

SMS Module — Complete Implementation Guide

Barangay Tisa Waste Management System

Using TextBee Android Gateway · P0 Cost with Unli-Text

Table of Contents

- 1. Module Overview
- 2. Use Cases
- 3. API Credentials (Already Provided)
- 4. TextBee Setup (Optional — Already Configured)
- 5. Files to Create/Update
- 6. Step-by-Step Implementation
- 7. Testing
- 8. Troubleshooting
- 9. Summary

1. Module Overview

The SMS Notification Feature sends automated SMS notifications to residents when their complaint status is updated. This ensures residents are informed of progress without unnecessary messages.

Provider	TextBee (Android Gateway)
Cost	P0 (uses your phone's unli-text promo)
Scope	SMS is sent only on complaint status updates (not on initial filing).

2. Use Cases

Use Case Name: Send Complaint Status SMS

Purpose: To send SMS notifications to residents when their complaint status changes from "Awaiting Review" to "Ongoing" or "Accomplished".

Actor/s: System (Automatic)

Input Parameters:

Parameter	Source	Description
Recipient Number	Complaint.ContactNumber	Resident's mobile number
Ticket Number	Complaint.TicketNumber	Unique complaint identifier
New Status	Complaint.Status	Updated complaint status

Output Parameters:

- SMS log record
- Success or error message

Pre-Condition:

- Complaint must have a valid Contact Number.
- Complaint status must change from one status to another.

Post-Condition:

- SMS is sent (or logged) to the resident.
- SMS log is recorded in the database.

Successful Scenario:

- Inspector/Admin updates complaint status.
- System detects status change.
- System sends SMS: "Barangay Tisa: Your complaint #[TicketNumber] is now [Status]. Thank you for your patience."
- System records SMS in the database.

Exception Scenario:

- No Contact Number:** SMS is skipped.
- Invalid number:** SMS fails, logged as "Failed".
- Daily limit reached:** SMS skipped.

Additional Remarks:

- SMS is sent automatically on status change.
- Daily limit: 50 SMS per day (database-based counter).
- SMS is NOT sent on initial complaint filing.

3. API Credentials (Already Provided)

These credentials are already configured and ready to use:

Credential	Value
TextBee API Key	txb_kwm4t1Qb7RrFrHrUTohLdlni7rfRESdN
TextBee Device ID	6a8070893005599046c02c7c
Provider	TextBee

These are already included in the appsettings.json file below. No additional setup is needed.

4. TextBee Setup (Optional — Already Configured)

You do **NOT** need to do any TextBee setup. The credentials above are already active and working. The Android phone with TextBee is already running. For your reference only (if you ever need to set up your own):

1. Install TextBee app on Android phone
2. Create account
3. Grant SMS permission
4. Copy API key from Settings → API
5. Copy Device ID from Devices

Skip this step. The system already has working credentials.

5. Files to Create/Update

File	Action	Purpose
Models/SmsLog.cs	CREATE	SMS log model
Services/ISmsService.cs	CREATE	SMS service interface
Services/SmsService.cs	CREATE	SMS service implementation
Controllers/ComplaintManagementController.cs	EDIT	Add SMS on status update
Controllers/SmsController.cs	CREATE	SMS log viewer
Views/Sms/Index.cshtml	CREATE	SMS log page
Data/ApplicationDbContext.cs	EDIT	Add SmsLogs DbSet
Views/Shared/_Layout.cshtml	EDIT	Add SMS Logs link
Program.cs	EDIT	Register SMS service
appsettings.json	EDIT	Add SMS settings (already has credentials)

6. Step-by-Step Implementation

Step 1: Create SmsLog Model

File: `Models/SmsLog.cs`

C#

```
using System.ComponentModel.DataAnnotations;

namespace TisaWasteManagement.Models
{
    public class SmsLog
    {
        [Key]
        public int SmsLogId { get; set; }

        [Required]
        [StringLength(11)]
        [Display(Name = "Recipient Number")]
        public string RecipientNumber { get; set; } = string.Empty;

        [Required]
        [StringLength(500)]
        [Display(Name = "Message")]
        public string Message { get; set; } = string.Empty;

        [StringLength(50)]
        [Display(Name = "Notification Type")]
        public string NotificationType { get; set; } = string.Empty;

        [Display(Name = "Reference ID")]
        public int? ReferenceId { get; set; }

        [StringLength(50)]
        [Display(Name = "Status")]
        public string Status { get; set; } = "Pending";

        [Display(Name = "Sent Date")]
        public DateTime SentDate { get; set; } = DateTime.Now;

        [StringLength(1000)]
        [Display(Name = "Response")]
        public string? Response { get; set; }

        [StringLength(100)]
        [Display(Name = "Sent By")]
        public string? SentBy { get; set; }
    }
}
```

Step 2: Add DbSet to ApplicationDbContext

File: `Data/ApplicationDbContext.cs`

Add this line with the other DbSet:

C#

```
public DbSet<SmsLog> SmsLogs { get; set; }
```

Step 3: Create SMS Service Interface

File: `Services/ISmsService.cs`

C#

```
using TisaWasteManagement.Models;

namespace TisaWasteManagement.Services
{
    public interface ISmsService
    {
        Task<bool> SendSmsAsync(string phoneNumber, string message, string notificationType, int? referenceId = null);
        Task<List<SmsLog>> GetLogsAsync(DateTime? startDate = null, DateTime? endDate = null);
    }
}
```

Step 4: Create SMS Service Implementation

File: `Services/SmsService.cs`

C#

```
using Microsoft.EntityFrameworkCore;
using System.Text;
using System.Text.Json;
using TisaWasteManagement.Data;
using TisaWasteManagement.Models;

namespace TisaWasteManagement.Services
{
    public class SmsService : ISmsService
    {
        private readonly ApplicationDbContext _context;
        private readonly IHttpClientFactory _httpClientFactory;
        private readonly IConfiguration _configuration;

        public SmsService(ApplicationDbContext context, IHttpClientFactory httpClientFactory, IConfiguration configuration)
        {
            _context = context;
            _httpClientFactory = httpClientFactory;
            _configuration = configuration;
        }
    }
}
```

```

{
    _context = context;
    _httpClientFactory = httpClientFactory;
    _configuration = configuration;
}

public async Task<bool> SendSmsAsync(string phoneNumber, string message, string notificationType, int? referenceId = null)
{
    try
    {
        // 1. Check daily limit (50 SMS per day) - DATABASE BASED
        if (!await CanSendSmsToday())
        {
            await LogSms(phoneNumber, message, notificationType, referenceId, "Failed", "Daily SMS limit reached (50/day).");
            return false;
        }

        // 2. Validate phone number
        if (!IsValidPhilippineNumber(phoneNumber))
        {
            await LogSms(phoneNumber, message, notificationType, referenceId, "Failed", "Invalid phone number.");
            return false;
        }

        // 3. Get TextBee credentials
        var apiKey = _configuration["SmsSettings:TextBeeApiKey"];
        var deviceId = _configuration["SmsSettings:TextBeeDeviceId"];

        if (string.IsNullOrEmpty(apiKey) || string.IsNullOrEmpty(deviceId))
        {
            await LogSms(phoneNumber, message, notificationType, referenceId, "Failed", "TextBee API key or Device ID not configured.");
            return false;
        }

        // 4. Format phone number to E.164 format (+639XXXXXXXX)
        var formattedNumber = FormatPhoneNumber(phoneNumber);

        if (string.IsNullOrEmpty(formattedNumber))
        {
            await LogSms(phoneNumber, message, notificationType, referenceId, "Failed", "Invalid phone number format.");
            return false;
        }

        var client = _httpClientFactory.CreateClient();

        // 5. Correct URL with device ID in path
        var url = $"https://api.textbee.dev/api/v1/gateway/devices/{deviceId}/send-sms";

        // 6. API key goes in HEADER (not body)
        client.DefaultRequestHeaders.Remove("x-api-key");
        client.DefaultRequestHeaders.Add("x-api-key", apiKey);

        // 7. Correct payload shape - recipients as array
        var payload = new
        {
            recipients = new[] { formattedNumber },
            message = message
        };

        var jsonPayload = JsonSerializer.Serialize(payload);
        var content = new StringContent(jsonPayload, Encoding.UTF8, "application/json");

        var response = await client.PostAsync(url, content);
        var responseContent = await response.Content.ReadAsStringAsync();

        if (response.IsSuccessStatusCode)
        {
            await LogSms(phoneNumber, message, notificationType, referenceId, "Sent", responseContent);
            return true;
        }

        await LogSms(phoneNumber, message, notificationType, referenceId, "Failed", $"HTTP {(int)response.StatusCode}: {responseContent}");
        return false;
    }
    catch (Exception ex)
    {
        await LogSms(phoneNumber, message, notificationType, referenceId, "Failed", ex.Message);
        return false;
    }
}

// Database-based daily limit check (survives app restarts)
private async Task<bool> CanSendSmsToday()
{
    var today = DateTime.Today;
    var tomorrow = today.AddDays(1);

    var count = await _context.SmsLogs
        .CountAsync(s =>
            s.SentDate >= today &&
            s.SentDate < tomorrow &&
            s.Status == "Sent");

    return count < 50;
}

private async Task LogSms(string phoneNumber, string message, string notificationType, int? referenceId, string status, string response)
{
    var log = new SmsLog
    {
        RecipientNumber = phoneNumber,
        Message = message,
        NotificationType = notificationType,
        ReferenceId = referenceId,
        Status = status,
        Response = response,
        SentDate = DateTime.Now,
        SentBy = "System"
    };
};

```

```

        _context.SmsLogs.Add(log);
        await _context.SaveChangesAsync();
    }

    private bool IsValidPhilippineNumber(string phoneNumber)
    {
        var cleaned = new string(phoneNumber.Where(char.IsDigit).ToArray());
        return cleaned.Length == 11 && cleaned.StartsWith("09");
    }

    private string FormatPhoneNumber(string phoneNumber)
    {
        if (string.IsNullOrEmpty(phoneNumber))
            return null;

        var cleaned = new string(phoneNumber.Where(char.IsDigit).ToArray());

        // 09171234567 -> +639171234567
        if (cleaned.Length == 11 && cleaned.StartsWith("09"))
        {
            return "+63" + cleaned.Substring(1);
        }

        // 639171234567 -> +639171234567
        if (cleaned.Length == 12 && cleaned.StartsWith("63"))
        {
            return "+" + cleaned;
        }

        return null;
    }

    public async Task<List<SmsLog>> GetLogsAsync(DateTime? startDate = null, DateTime? endDate = null)
    {
        var query = _context.SmsLogs.AsQueryable();

        if (startDate.HasValue)
            query = query.Where(s => s.SentDate >= startDate.Value.Date);

        if (endDate.HasValue)
            query = query.Where(s => s.SentDate <= endDate.Value.Date.AddDays(1));

        return await query
            .OrderByDescending(s => s.SentDate)
            .ToListAsync();
    }
}

```

Step 5: Register Service in Program.cs

File: `Program.cs`

C#

```

// Add after builder.Services.AddControllersWithViews();

builder.Services.AddHttpClient();
builder.Services.AddScoped<ISmsService, SmsService>();

```

Step 6: Add SmsSettings to appsettings.json

File: `appsettings.json`

JSON

```

{
  "SmsSettings": {
    "Provider": "TextBee",
    "TextBeeApiKey": "txb_kwm4t1Qb7RrFrHrUTohLd1ni7rfRESdN",
    "TextBeeDeviceId": "6a8070893005599046c02c7c"
  }
}

```

Step 7: Update ComplaintManagementController

File: `Controllers/ComplaintManagementController.cs`

Add this code in the Resolve POST method:

C#

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Resolve(ResolveComplaintViewModel viewModel)
{
    // ... existing validation code ...

    // Store old status for comparison
    var oldStatus = complaintToUpdate.Status;

    // Update the complaint
    complaintToUpdate.Status = viewModel.SelectedStatus;
    complaintToUpdate.UpdatedDate = DateTime.Now;

    _context.Entry(complaintToUpdate).State = EntityState.Modified;
    await _context.SaveChangesAsync();

    // SEND SMS ON STATUS UPDATE
    // This will trigger when: Awaiting Review -> Ongoing -> Accomplished
    if (oldStatus != viewModel.SelectedStatus &&
        !string.IsNullOrEmpty(complaintToUpdate.ContactNumber) &&
        viewModel.SelectedStatus != "Awaiting Review")
    {
        try
        {
            var smsService = HttpContext.RequestServices.GetRequiredService<ISmsService>();
            var message = $"Barangay Tisa: Your complaint #{complaintToUpdate.TicketNumber} is now {viewModel.SelectedStatus}. Thank you for your patience.";
            await smsService.SendSmsAsync(complaintToUpdate.ContactNumber, message, "ComplaintUpdate", complaintToUpdate.ComplaintId);
        }
        catch (Exception ex)
        {
            Console.WriteLine($"SMS sending failed: {ex.Message}");
        }
    }

    TempData["Success"] = $"Complaint #{complaintToUpdate.TicketNumber} status updated to {viewModel.SelectedStatus} successfully!";
    return RedirectToAction("Index");
}
```

Step 8: Create SmsController

File: `Controllers/SmsController.cs`

C#

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using TisaWasteManagement.Data;
using TisaWasteManagement.Helpers;
using TisaWasteManagement.Services;

namespace TisaWasteManagement.Controllers
{
    [RequireStaffRole("Admin")]
    public class SmsController : Controller
    {
        private readonly ISmsService _smsService;
        private readonly IConfiguration _configuration;
        private readonly ApplicationDbContext _context;

        public SmsController(ISmsService smsService, IConfiguration configuration, ApplicationDbContext context)
        {
            _smsService = smsService;
            _configuration = configuration;
            _context = context;
        }

        public async Task<IActionResult> Index(DateTime? startDate, DateTime? endDate)
        {
            var logs = await _smsService.GetLogsAsync(startDate, endDate);

            // Stats
            ViewBag.TotalSent = await _context.SmsLogs.CountAsync(s => s.Status == "Sent");
            ViewBag.TotalFailed = await _context.SmsLogs.CountAsync(s => s.Status == "Failed");
            ViewBag.Provider = _configuration["SmsSettings:Provider"] ?? "TextBee";

            return View(logs);
        }
    }
}
```

Step 9: Create SMS Log View

File: `Views/Sms/Index.cshtml`

HTML

```

@model List<TisaWasteManagement.Models.SmsLog>

@{
    ViewData["Title"] = "SMS Logs";
    Layout = "~/Views/Shared/_Layout.cshtml";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <div>
        <h1 class="mb-0">SMS Logs</h1>
        <p class="text-muted">View SMS notifications sent to residents.</p>
    </div>
    <span class="badge bg-success">
        @ViewBag.Provider
    </span>
</div>

<!-- Stats -->
<div class="row g-3 mb-4">
    <div class="col-md-4">
        <div class="card bg-success text-white">
            <div class="card-body">
                <h6>Total Sent</h6>
                <h3>@ViewBag.TotalSent</h3>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card bg-danger text-white">
            <div class="card-body">
                <h6>Total Failed</h6>
                <h3>@ViewBag.TotalFailed</h3>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card bg-info text-white">
            <div class="card-body">
                <h6>Provider</h6>
                <h3>@ViewBag.Provider</h3>
            </div>
        </div>
    </div>
</div>

@if (!Model.Any())
{
    <div class="text-center text-muted py-5">
        <i class="bi bi-envelope" style="font-size: 3rem;"></i>
        <p class="mt-3">No SMS logs found.</p>
        <p class="text-muted small">SMS will appear here when complaint statuses are updated.</p>
    </div>
}
else
{
    <div class="table-responsive">
        <table class="table table-striped table-hover">
            <thead>
                <tr>
                    <th>Date</th>
                    <th>Number</th>
                    <th>Message</th>
                    <th>Type</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var item in Model)
                {
                    <tr>
                        <td>@item.SentDate.ToString("MMM d, yyyy h:mm tt")</td>
                        <td>@item.RecipientNumber</td>
                        <td>@item.Message</td>
                        <td>
                            <span class="badge bg-info">
                                @item.NotificationType
                            </span>
                        </td>
                        <td>
                            <span class="badge @(item.Status == "Sent" ? "bg-success" : "bg-danger")">
                                @item.Status
                            </span>
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
}

```

Step 10: Add to Sidebar Navigation

File: `Views/Shared/_Layout.cshtml`

HTML

```

<!-- Admin Side -->
<a class="nav-link" asp-area="" asp-controller="Sms" asp-action="Index">
    <i class="bi bi-envelope"></i> SMS Logs
</a>

```

Step 11: Create Database Migration

BASH

```
dotnet ef migrations add AddSmsLogsTable
dotnet ef database update
```

Step 12: Build and Run

BASH

```
dotnet build
dotnet run
```


7. Testing

Test: Status Update SMS

Step	Action	Expected Result
1	Login as Admin/Inspector	Dashboard displays
2	Find complaint with status "Awaiting Review"	Complaint listed
3	Change status to "Ongoing"	Status updated
4	Check SMS Logs	Shows "Sent" with message
5	Check your phone	SMS received

Test: No SMS on Complaint Filed

Step	Action	Expected Result
1	File a new complaint	Complaint saved
2	Check SMS Logs	NO SMS logged for filing

8. Troubleshooting

Issue	Fix
No SMS received	Check TextBee app is open and running on the Android phone
SMS shows "Failed"	Check API key and Device ID in appsettings.json
SMS shows "Failed"	Check phone has internet connection
SMS shows "Failed"	Check SIM has unli-text promo
Daily limit reached	Wait until next day
App restart resets counter	Fixed with database-based counter

9. Summary

Feature	Status
SMS on status update	✓
SMS on complaint filed	✗ (removed to save credits)
SMS logs stored in database	✓
Database-based daily limit (50/day)	✓
SMS Logs page for Admin	✓
TextBee integration	✓
Cost	₱0 (unli-text)

SMS Messages

Trigger	Message
Status Updated	Barangay Tisa: Your complaint #[TicketNumber] is now [Status]. Thank you for your patience.

Quick Setup Checklist

- ☐ Copy SmsLog.cs to Models/
- ☐ Add SmsLogs DbSet to ApplicationDbContext.cs
- ☐ Copy ISmsService.cs to Services/
- ☐ Copy SmsService.cs to Services/
- ☐ Update ComplaintManagementController.cs
- ☐ Copy SmsController.cs to Controllers/
- ☐ Copy Index.cshtml to Views/Sms/
- ☐ Add SMS Logs link to _Layout.cshtml
- ☐ Update Program.cs (add SMS service)
- ☐ Update appsettings.json (add SMS settings)
- ☐ Run migration
- ☐ Build and run